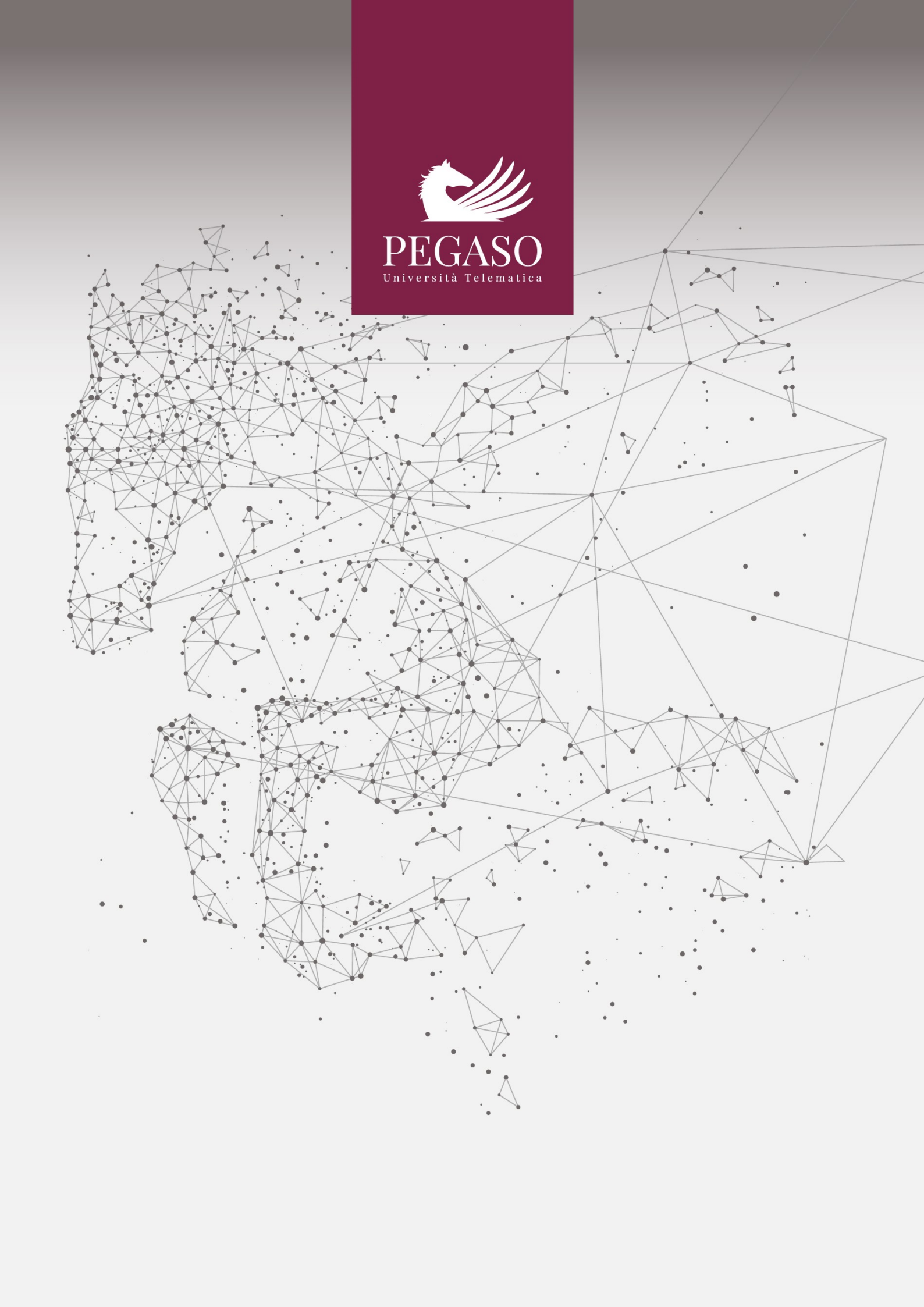

PEGASO
Università Telematica

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Indice

1. INTRODUZIONE 3

2. OPERAZIONI MATEMATICHE SULLE VARIABILI 4

3. OPERAZIONI CON TIPI MISTI E CONVERSIONE DEI TIPI DI DATO 7

4. FORMATTAZIONE DELL'OUTPUT 9

BIBLIOGRAFIA 14

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

1. Introduzione

In questa dispensa di accompagnamento alla video-lezione, si introducono le operazioni matematiche sulle variabili e la formattazione dell'output in ambiente Python. Verranno fornite anche indicazioni sulla precedenza degli operatori in Python, sulle operazioni con tipi misti di dato e sulla conversione dei tipi di dato.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2. Operazioni matematiche sulle variabili

Nella scrittura di programmi, molto spesso è necessario ricorrere ad **espressioni matematiche** che permettono di eseguire i calcoli necessari per la risoluzione di un problema e di ottenere un risultato.

Le espressioni matematiche coinvolgono un **operatore matematico** ed uno o più **operandi** (variabili e/o letterali numerici). Gli operatori matematici più comuni sono quelli aritmetici:

- addizione +
- sottrazione -
- moltiplicazione *
- divisione.

In merito all'operazione di divisione, è opportuno menzionare che Python prevede due tipi di operatore diverso in base al tipo di operazione. L'operatore / per la divisione in virgola mobile e l'operatore // per la divisione intera che restituisce un risultato intero. Nel codice sorgente che segue vengono riportate le istruzioni di esempio corrispondenti ai due tipi di divisione:

```
a=5/3
b=5//3

print("a =",a)
print("b =",b)
```

Il risultato che verrà visualizzato a schermo dopo l'esecuzione del codice è:

```
a = 1.6666666666666667
b = 1
```

A titolo di esempio, si riporta il codice sorgente di un semplice esercizio di calcolo del costo totale di un servizio noti il costo orario del servizio e il numero di ore di servizio. Si ipotizza un costo orario di € 15,00 e un numero di ore pari a 3,5. Il costo totale del servizio, pari al prodotto del costo orario e del numero delle ore, verrà memorizzato nella variabile *totale* e, successivamente, visualizzato a schermo.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
costo_orario = 15.0
ore = 3.5
totale = costo_orario*ore
print("Il costo totale è pari a",totale)
```

Il risultato dell'elaborazione sarà il seguente:

```
Il costo totale è pari a 52.5
```

Molto interessante è la selezione dell'operatore corretto di divisione per il calcolo di medie. Si ipotizzi di ricevere in input tre voti e di calcolarne la media. L'operatore corretto sarà l'operatore / di divisione in virgola mobile. Segue il codice sorgente del calcolo di esempio di una media.

```
votol = float(input("Inserire il primo voto: "))
voto2 = float(input("Inserire il secondo voto: "))
voto3 = float(input("Inserire il terzo voto: "))

media = (votol+voto2+voto3) / 3.0

print("La media dei voti è",media)
```

Il risultato dell'elaborazione è il seguente:

```
Inserire il primo voto: 8
Inserire il secondo voto: 8
Inserire il terzo voto: 7
La media dei voti è 7.666666666666667
```

Altri operatori matematici di rilievo e molto utilizzati sono:

- l'**operatore esponente** ** (doppio asterisco) che eleva un numero ad una potenza, es. x**y = x^y;
- l'**operatore resto** % (simbolo di percentuale) che esegue la divisione tra due operandi e restituisce il resto di tale divisione, es. 4%2 = 0 , 5%2 = 1. Tipicamente l'operatore resto viene utilizzato per convertire misure di tempo e distanza o per verificare se un numero è pari (risultato dell'operazione di resto con secondo operando 2 pari a zero) o dispari (risultato dell'operazione di resto con secondo operando 2 pari a uno).

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Un esempio molto interessante dell'utilizzo dell'operatore resto è quello relativo alle conversioni delle misure di tempo. Nell'esempio che segue, si richiede in input il numero di secondi da convertire in ore, minuti e secondi. Data la corrispondenza 1 ora – 3600 secondi, il numero di ore sarà dato dal resto della divisione tra il numero totale dei secondi e 3.600. Il numero dei minuti, invece, sarà dato dal resto della divisione tra il numero dei minuti rimanenti dopo la trasformazione in ore (total_seconds // 60) e 60. Infine, il numero dei secondi rimanenti dopo le trasformazioni precedenti, sarà dato dal resto della divisione intera tra il numero totale di secondi e 60.

Il codice sorgente che segue permette di effettuare la conversione appena descritta:

```
# Riceve in ingresso un numero di secondi.
total_seconds = float(input('Inserire il numero di secondi: '))

# Calcola il numero di ore.
hours = total_seconds // 3600

# Calcola il numero di minuti rimanente.
minutes = (total_seconds // 60) % 60

# Calcola il numero di secondi rimanente.
seconds = total_seconds % 60

# Visualizza il risultato.
print('Ecco il tempo espresso in ore, minuti e secondi:')
print('Ore:', hours)
print('Minuti:', minutes)
print('Secondi:', seconds)
```

Il risultato dell'elaborazione sarà il seguente:

```
Inserire il numero di secondi: 12357
Ecco il tempo espresso in ore, minuti e secondi:
Ore: 3.0
Minuti: 25.0
Secondi: 57.0
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

2.1 Precedenza degli operatori in Python

I criteri di precedenza di risoluzione degli operatori in Python sono i seguenti in ordine:

1. le operazioni racchiuse tra parentesi;
2. l'operazione di elevazione a potenza;
3. la moltiplicazione, la divisione (entrambi gli operatori / e //) e il calcolo del resto della divisione intera;
4. l'addizione e la sottrazione.

A parità di precedenza dell'operatore, si procede in ordine di apparizione da sinistra a destra dell'operatore stesso.

3. Operazioni con tipi misti e conversione dei tipi di dato

Il tipo di dato risultante da un'operazione matematica dipende dai tipi di dato degli operandi. In particolare, il risultato di un'operazione tra due operandi di tipo intero int sarà intero int. Stessa cosa per le operazioni tra due operandi di tipo float, il cui risultato sarà float.

Nelle operazioni con tipi misti int e float, l'operando di tipo intero int viene temporaneamente convertito in float e il risultato dell'operazione sarà float. È fondamentale mettere in evidenza che la conversione di tipo da float a int causa l'eliminazione della parte decimale.

Il risultato dell'istruzione *ris = 3*7* sarà, infatti, diverso da quello dell'istruzione *ris = 3*7.0*. Il codice sorgente per la dimostrazione è:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
ris = 3 * 7
print(type(ris))

ris = 3 * 7.0
print(type(ris))
```

Il risultato sarà infatti diverso per tipo di dato:

```
<class 'int'>
<class 'float'>
```

Per convertire il tipo di dato di una variabile, Python mette a disposizione delle funzioni di casting:

- *int(x)* converte la variabile *x* da float o da str ad intero int;
- *float(x)* converte la variabile *x* da int o da str a float.

Ad esempio, il seguente codice sorgente:

```
a = 5.4
b = int(a)
print(b)
```

visualizzerà a schermo 5.

Per le istruzioni particolarmente lunghe e, quindi, non visualizzabili sullo schermo nell'editor di testo, Python mette a disposizione il **carattere di continuazione di riga ** . Tale carattere permette di interrompere l'istruzione e riportarla su più linee come ad esempio:

```
print("Mi chiamo",\
      nome)
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

4. Formattazione dell'output

Al fine di migliorare la visualizzazione dei risultati delle operazioni sui dati, è possibile modificare i parametri della funzione **print**(). In interactive mode, se si scrive print(appare una piccola finestra come di seguito riportato:

```
print(
      print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)
```

Tale finestra riporta, in ordine, i parametri da passare alla funzione print al fine di visualizzare a schermo dei dati.

In particolare, **value** è il parametro (es. stringa) da visualizzare a schermo. I tre puntini di sospensione … stanno a significare la possibilità di aggiungere altri parametri per la visualizzazione a schermo.

Il parametro **sep=' '** sta ad indicare che tra i parametri value da visualizzare a schermo, la funzione print inserisce degli spazi. Per modificare il separatore, ad esempio con l'inserimento del punto e virgola, dovremo riportare nella chiamata a print:

```
data = 15
print("oggi è il giorno",data, sep = ';')
```

Il risultato di tale operazione è:

```
oggi è il giorno;15
```

L'argomento speciale **end = '\n'** indica alla funzione print di inserire alla fine della visualizzazione il carattere \n di new line (a capo). Per modificarlo, ad esempio volendo richiamare più volte la funzione print andando a capo solo dopo l'ultima chiamata dovremo scrivere nel codice sorgente:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print("ieri", end = ' ')
print("oggi", end = ' ')
print("domani", end = '\n')
```

che produrrà il seguente risultato:

```
ieri oggi domani
```

Il parametro **file = sys.stdout** indica che print dovrà visualizzare gli argomenti a schermo (che rappresenta lo standard output - stdout), mentre il parametro **flush = False** specifica se l'output deve essere visualizzato a schermo carattere per carattere (True) o bufferizzato (l'insieme dei parametri passata a print tutti insieme) (False). Il valore di default per flush è False.

4.1 Formattazione delle stringhe

Sempre per una migliore resa di visualizzazione dell'output, nei **letterali di stringa** (stringhe nel codice sorgente) possono essere inseriti dei caratteri speciali denominati **caratteri di escape**. Tali caratteri, preceduti dal backslash \ vengono trattati come comandi inclusi nella stringa e permettono di formattare la visualizzazione.

I caratteri di escape più diffusi sono:

- \n newline;
- \t tabulazione orizzontale;
- \' stampa virgoletta singola;
- \" stampa virgoletta doppia;
- \\ stampa una barra inversa.

Ad esempio, se volessimo visualizzare i giorni della settimana dal Lunedì al Sabato secondo lo schema di una tabella da 2 righe e 3 colonne con una sola chiamata alla funzione print, potremmo utilizzare il carattere di tabulazione orizzontale e il carattere di new line nel seguente modo:

```
print("Lun\tMar\tMer\nGio\tVen\tSab")
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

ottenendo il seguente risultato:

```
Lun     Mar     Mer
Gio     Ven     Sab
```

Sempre in merito alle stringhe, l'operatore + messo a disposizione da Python realizza la **concatenazione di stringhe**. Nell'estratto del codice sorgente che segue, si mostrano tre diverse modalità per ottenere la visualizzazione a schermo della frase completa:

```
s = "Domani sarà proprio " + "una bella giornata"
print(s)
print("Domani sarà proprio " + "una bella giornata")
print("Domani sarà proprio", "una bella giornata")
```

A schermo verrà visualizzato, quindi:

```
Domani sarà proprio una bella giornata
Domani sarà proprio una bella giornata
Domani sarà proprio una bella giornata
```

4.2 Formattazione dei numeri

È possibile decidere il formato dei numeri nella visualizzazione a schermo usando la funzione di Python built-in **format**(). La funzione format( ) ha due argomenti: il primo è il **valore numerico da formattare**, mentre il secondo è lo **specificatore di formato**. La funzione format dati i due argomenti restituirà una **stringa** contenente il numero formattato.

Lo specificatore di formato tipicamente include la **precisione** e il **tipo di dato** e può essere usato per indicare la notazione scientifica, i separatori delle migliaia, la percentuale, ecc. Poiché Python è un linguaggio di programmazione sviluppato negli Stati Uniti, la formattazione standard di default prevede separatori delle migliaia e dei decimali nella forma statunitense (anche la funzione format() utilizza questo standard). Per formattare le stringhe secondo altri formati bisognerà adoperare il modulo **locale**.

Verranno ora presentati alcuni esempi di formattazione dei numeri. Nel primo esempio, il numero 12345.6780000009 verrà stampato, rispettivamente, con 3 cifre decimali, utilizzando la notazione scientifica e con 2 cifre decimali utilizzando la notazione scientifica:

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
print(format(12345.6780000009, ".3f"))     #3 cifre decimali
print(format(12345.6780000009, "e"))       # notazione scientifica
print(format(12345.6780000009, ".2e"))    # notazione scientifica
                                           con due cifre decimali
```

Il risultato a schermo dell'esecuzione del codice sorgente sarà:

```
12345.678
1.234568e+04
1.23e+04
```

Nel secondo esempio, il numero 12345.6780000009 verrà stampato, rispettivamente, con: il separatore delle migliaia, 2 cifre decimali ed occuperà 15 caratteri totali; senza il separatore delle migliaia, 2 cifre decimali ed occuperà 15 caratteri totali.

```
print(format(12345.6780000009, "15,.2f"))
#15 si riferisce al numero di spazi che
#deve occupare il numero_allineamento a destra

print(format(12345.6780000009, "15.2f"))
#15 si riferisce al numero di spazi che deve
#occupare il numero_allineamento a destra
#MA senza virgole delle migliaia
```

Il risultato a schermo dell'esecuzione del codice sorgente sarà:

```
     12,345.68
      12345.68
```

Nel terzo e ultimo esempio, si riportano i seguenti casi:

1. numero espresso in percentuale con due cifre decimali;
2. numero intero senza separatore delle migliaia;
3. numero intero con separatore delle migliaia;
4. numero intero senza separatore delle migliaia in uno spazio di 10 caratteri totali;
5. numero intero con separatore delle migliaia in uno spazio di 10 caratteri totali.

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

```
# utile esempio per percentuale
print(format(0.5,'.2%'))

# senza separatore migliaia
print(format(123456,'d'))

#con separatore migliaia
print(format(123456,',d'))

#senza separatore in spazio da 10
print(format(123456,'10d'))

#con separatore in spazio da 10
print(format(123456,'10,d'))
```

Il risultato a schermo dell'esecuzione del codice sorgente sarà:

```
50.00%
123456
123,456
    123456
   123,456
```

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).

Bibliografia

- Gaddis, T. (2016). Introduzione a Python. Ediz. mylab. Con espansione online. Pearson.
- Python (2021). https://www.python.org/

Attenzione! Questo materiale didattico è per uso personale dello studente ed è coperto da copyright. Ne è severamente vietata la riproduzione o il riutilizzo anche parziale, ai sensi e per gli effetti della legge sul diritto d'autore (L. 22.04.1941/n. 633).